

Esquemas de Tradução para Bison/YACC

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

18 de março de 2026

Esquemas de tradução com Bison e Lex/Flex

- Neste conjunto de slides será mostrado como é possível implementar esquemas de tradução com analisadores ascendentes gerados pelo Bison/Yacc. Para isso, vamos adotar a seguinte sequência:
 - Recordar o que é um esquema de tradução.
 - Estudar como os atributos dos símbolos gramaticais são representados no Bison/Yacc.
 - Estudar como o analisador léxico gerado pelo Flex/Lex pode ter acesso aos atributos dos símbolos gramaticais.

Esquemas de tradução - recordação

- Um esquema de tradução é um formalismo de tradução dirigida por sintaxe que inclui as ações semânticas em algum lugar do lado direito das produções, podendo as ações ocorrerem antes, entre ou após os símbolos gramaticais da produção.
- As ações semânticas ocorrem entre chaves, por exemplo:

$$D \rightarrow T\{L.tipo = T.tipo\}L$$

Representação de atributos no Bison/Yacc

- Por ser uma ferramenta de propósito geral, o Bison precisou propor um modo genérico de permitir o uso de atributos durante o *parsing* (tradução dirigida por sintaxe).
- Para isso, duas pilhas são utilizadas pela ferramenta: uma **pilha sintática** e uma **pilha semântica**.
 - A pilha sintática é aquela usada pelo *parser* durante a análise sintática. Por exemplo, dada uma produção $A \rightarrow BCD$, na pilha sintática D aparece no topo da pilha, seguido C e B. No momento da redução, BCD são desempilhados e substituídos no topo da pilha por A .
 - A pilha semântica é uma pilha que armazena atributos dos símbolos gramaticais que ficam na pilha sintática.
- O programador pode usar ações semânticas para inserir valores de atributos na pilha semântica que sejam correspondentes aos símbolos gramaticais da pilha sintática.

Representação de atributos no Bison/Yacc

- O Bison permite que se acesse os atributos na pilha semântica correspondentes aos símbolos gramaticais que estão atualmente no topo da pilha sintática. Sejam $X_1, X_2, \dots, X_n$ os símbolos na parte superior da pilha sintática (X_n está no topo). Os seus valores de atributos podem ser acessados utilizando-se as seguintes denominações, respectivamente: $\$1, \$2, \dots, \$n$.
- Se $X_1, X_2, \dots, X_n$ são reduzidos a um não-terminal A o atributo de A pode ser referido por $\$A$.
- Exemplo. Suponha um trecho de um esquema de tradução que calcula os valores de uma expressão aritmética:
 $term_1 \rightarrow term_2 * factor\{term_1.val = term_2.val * factor.val\}$.
No Bison, esse trecho poderia ser representado por:

$$term : term * factor\{\$A = \$1 * \$3;\}$$

a ação semântica entre chaves é escrita na linguagem C.

O tipo de dados da pilha semântica

- Em um esquema de tradução podem ocorrer atributos de tipos de dados distintos, por exemplo, um atributo pode ser do tipo `char`, outro pode ser do tipo `double`, outro do tipo `struct`, etc.
- Como atribuir valores a atributos com tipos distintos na pilha semântica?

O tipo de dados da pilha semântica

- Em um esquema de tradução podem ocorrer atributos de tipos de dados distintos, por exemplo, um atributo pode ser do tipo `char`, outro pode ser do tipo `double`, outro do tipo `struct`, etc.
- Como atribuir valores a atributos com tipos distintos na pilha semântica?
- A resposta para essa questão é que o Bison permite ao programador substituir o tipo `int` (tipo padrão) da pilha semântica por um tipo `union` que corresponde a várias alternativa de tipos.

Definição do tipo de dados da pilha semântica

- Para criar a `union`, o programador deve definir os tipos de atributos para os tokens e para os terminais. Esses tipos podem ser os tipos básicos da linguagem C, ou tipos criados pelo programador, nomeados ou não com `typedef`.
- Uma vez definido o conjunto de tipos dos atributos, eles são agrupados em uma estrutura tipo `union` da linguagem C. O Bison tem a diretiva `%union` para permitir especificar essa `union`.
- O que fazer se um símbolo da gramática tiver mais de um atributo?

Definição do tipo de dados da pilha semântica

- Para criar a `union`, o programador deve definir os tipos de atributos para os tokens e para os terminais. Esses tipos podem ser os tipos básicos da linguagem C, ou tipos criados pelo programador, nomeados ou não com `typedef`.
- Uma vez definido o conjunto de tipos dos atributos, eles são agrupados em uma estrutura tipo `union` da linguagem C. O Bison tem a diretiva `%union` para permitir especificar essa `union`.
- O que fazer se um símbolo da gramática tiver mais de um atributo? Nesse caso, é necessário criar um tipo `struct` com um campo para cada atributo. Um campo para essa `struct` deve ser incluído na `union` que corresponderá ao tipo da pilha semântica.

Exemplo

- A seguir apresentaremos um exemplo bem completo de como usar a pilha semântica.
- Criaremos um esquema de tradução para gerar árvores abstratas para programas escritos em uma linguagem de programação muito simples (SimpleLang) - ver seção 2.8.2 do livro-texto.

Exemplo

- A gramática da SimpleLang é mostrada abaixo.

```
program → block
block → '{' stmts '}'
stmts → stmt stmts | ε
stmt → expr ';'
      | IF '(' expr ')' stmt
      | WHILE '(' expr ')' stmt
      | DO stmt WHILE '(' expr ')'
      | block
expr → rel '=' expr | rel
rel → rel '<' add | rel LEQ add | add
add → add '+' term | term
term → term '*' factor | factor ;
factor → '(' expr ')' | NUM
```

Exemplo

- Os nós internos da árvores abstrata correspondem a operadores.
- Um nó folha corresponde a um número (token NUM) e deve ter como atributo o lexema do token encontrado (i.e. a string de dígitos correspondente ao número).
- Precisamos, portanto, definir o tipo de dados que forma um nó da árvore abstrata. Isso é mostrado no próximo slide.

Representação de atributos no Bison/Yacc

Exemplo

- O código a seguir está armazenado no arquivo tipos.h e contém a descrição de um nó da árvore abstrata.

```
1  #ifndef _tipos_h
2  #define _tipos_h
3
4  typedef enum{lstStmt, If, While, Do, Num, Mais, Mult, Menor,
5             Igual, MenorIgual} TespecieOperador;
6  //Definicao de um no da arvore abstrata.
7  typedef struct operador{
8      TespecieOperador tipoOperador; // while, if, etc
9      int linha;
10     char* lexema; // utilizado para armazenar lexema de NUM
11     struct operador* filho1;
12     struct operador* filho2;
13 } Toperador;
14
15 #endif
```

Exemplo

- Criamos um único tipo (Toperador) para representar todos os nós da árvore.
- Para diferenciar o tipo de nó (operador / nó folha), usamos uma enumeração da linguagem C. Um nó pode armazenar, um de vários operadores: `1stStmt`, `IF`, etc, ou pode armazenar um operando (`Num`). A struct que representa um nó tem um campo cujo tipo é essa enumeração para identificar o tipo de nó da árvore abstrata.
- Há um campo para indicar a linha do programa de entrada correspondente ao operador/ operando.
- Há um ponteiro para armazenar o lexema, caso o nó seja correspondente a um número.
- Por fim, a estrutura de um nó contém dois ponteiros para o próprio tipo de dados Toperador que são ponteiros para os possíveis dois filhos do nó.

Representação de atributos no Bison/Yacc

Exemplo

As linhas a seguir correspondem a seção de prólogo do Bison:

```
1  %{
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <strings.h>
5  #include "tipos.h"
6  extern char * yytext;
7  extern int yylex();
8  extern int numLinha;
9  extern FILE* yyin;
10 extern int erroOrigem;
11 void yyerror( char const *s);
12 Toperador* CriaOperador(TespecieOperador tipoOperador,
13 int linha, Toperador* filho1, Toperador* filho2,
14 char* lexema);
15 void percorreArvore(Toperador* raiz);
16 void obtemEspecieNoEnumLin(Toperador* no, char* nomeOperador);
17 Toperador* raiz;
18 char nomeOperador[200];
19 %}
```

Exemplo

- A função:

```
1  Toperador* CriaOperador(TespecieOperador tipoOperador ,  
2  int linha, Toperador* filho1, Toperador* filho2,  
3  char* lexema);
```

recebe como parâmetros um valor da enumeração correspondente ao tipo de nó, o número da linha no programa fonte, os ponteiros para as duas subárvores filhas, podendo ser ambos iguais a NULL e um ponteiro para o lexema de NUM (ou NULL).

- A função tenta alocar espaço para uma struct correspondente ao nó e insere os parâmetros de entrada nos campos correspondentes à struct do tipo Toperador criada.
- A função retorna um ponteiro para o nó que ela cria, ou NULL, caso a criação do nó tenha falhado.

Exemplo

- A seguir, mostramos como podemos definir a union que corresponde ao tipo da pilha semântica para o exemplo:

```
1  %union{  
2      int nlinha;  
3      Toperador* Tpont;  
4      char *numLexema;  
5  }
```

- Essa declaração ocorre na seção de declaração de produções no Bison, logo após "%}"
- Nesse caso, três tipos distintos podem ocorrer na pilha semântica: um int um ponteiro para nó (Toperardor*) ou um um ponteiro para char.

Exemplo

- Para definir os tipos dos atributos correspondentes aos tokens, utiliza-se a palavra-chave `%token` antes da sequência de tokens e, coloca-se o tipo (campo da union) de uma sequência de tokens entre '`<`' e '`>`':

```
%token IF WHILE DO LEQ <numLexema> NUM
```

- A declaração acima indica que os tokens IF, WHILE, DO e LEQ não possuem atributos associados, mas o token NUM possui e o tipo é o tipo `numLexema` que é um ponteiro para `char`.

Exemplo

- Para definir os tipos dos atributos correspondentes aos não-terminais utiliza-se a palavra-chave `%type` antes da sequência de não-terminais e, coloca-se o tipo (campo da union) de uma sequência de não-terminais entre '`<`' e '`>`', antes da sequência:

```
%type<Tpont> block stmts stmt expr rel add term factor
```

- A declaração acima indica os tipos de dados dos atributos correspondentes aos não-terminais são ponteiros para nós da árvore abstrata, representados por `Tpont` que é, na verdade, um `Toperador` *. Veja a union mostrada acima.

Exemplo

- Apresentamos nos próximos slides o esquema de tradução para construção da árvore abstrata para um dado programa de entrada escrito em SimpleLang.
- Mostramos em cada slide as produções e as ações semânticas para um único não-terminal. Assim, podemos concentrar a atenção em um único não terminal e nas produções em que ele aparece do lado esquerdo dessas produções.
- Como o esquema de tradução será usado com o Bison, que faz uma análise sintática ascendente, iniciaremos com o não terminal `factor`, pois todos os nós-folhas correspondem ao token `NUM` e `NUM` é reduzido a `factor`.

Ações semânticas correspondentes às factor-produções

Exemplo

- As factor-produções são as seguintes:

```
factor : '(' expr ')'
      | NUM
      ;
```

- Na primeira produção, a ação semântica consiste em copiar o atributo \$2 de expr (ponteiro para a subárvore de expressão) para o atributo \$\$ de factor após a redução ($\{\$ \$ = \$2\}$).
- Na segunda produção, a função construtora é chamada para criar um nó-folha do tipo NUM, colocar NULL nos seus campos filho1 e filho2 e armazenar o lexema de NUM nesse nó. O endereço do nó é atribuído ao atributo \$. de factor durante a redução. As produções e as respectivas ações semânticas são apresentadas a seguir.

```
factor  : '(' expr ')' {$ $ = $2;}
      | NUM {$ $ = CriaOperador(Num, numLinha, NULL, NULL, $1);}
      ;
```

Inserção do atributo lexema (yytext) na pilha semântica

- Repare na segunda factor-produção e na chamada à função construtora CriaOperador:

```
| NUM {$$=CriaOperador(Num, numLinha, NULL, NULL, $1);}
```

O último parâmetro da chamada é \$1, que corresponde ao lexema de NUM (uma cópia de yytext). Esse ponteiro (numLexema) é inserido na pilha semântica pelo analisador léxico (yylex()), conforme mostrado nas seguintes linhas do arquivo de entrada para o Flex:

```
{DIGIT}+ {  
    yylval.numLexema=(char*)malloc(strlen(yytext)+1);  
    if(yylval.numLexema){  
        strcpy(yylval.numLexema,yytext);  
        return(NUM);  
    }  
    printf("Nao ha memoria para gerar a arvore.\n");  
    exit(1); }
```

Inserção do atributo lexema (`yylval`) na pilha semântica

- Repare que o analisador léxico tem acesso à pilha semântica por meio da variável `yylval` que é do mesmo tipo de dados especificado na diretiva `%union` do Bison.
- a diretiva `%union` faz com que o Bison defina um tipo denominado `YYSTYPE` em C para definir a `union` da diretiva. Quando o Bison é chamado com a opção `-d`, ele gera um arquivo de cabeçalho (`.h`) que contém essas definições e as atribuições de constantes inteiras positivas aos tokens. Esse arquivo `.h` pode ser incluído no arquivo de entrada para o Flex, através de um comando `#include`.
- No caso específico, quando o analisador léxico detecta a ocorrência de uma string que casa com `{DIGIT}+` na entrada, antes de retornar o token `NUM`, ele aloca espaço para uma string do tamanho da string apontada por `yylval` (lexema) e copia a string em `yylval` no campo `numLexema` da `union` `yylval`.

Ações semânticas correspondentes às term-produções

- As term-produções são as seguintes:

```
term : term '*' factor
      | factor
      ;
```

- Na primeira produção, a função `CriaOperador` é chamada para criar um nó para o operador `'*'` (`Mult` na enumeração) e inserir os ponteiros para as subárvores correspondentes a `term` (`term2`) e `factor` como filhos desse nó. Esses ponteiros se encontram, respectivamente, nas posições `$1` e `$3` da pilha semântica.
- Na segunda produção, o ponteiro para subárvore correspondente à `factor` é copiado para o atributo `$$` de `term`.

```
term: term '*' factor {
        $$=CriaOperador(Mult, numLinha, $1,$3, NULL);
    }
    | factor {$$=$1;}
    ;
```

Ações semânticas correspondentes às add-produções

- As add-produções são as seguintes:

```
add : add '+' term
    | term
    ;
```

- Na primeira produção, a função `CriaOperador` é chamada para criar um nó para o operador '+' (Mais na enumeração) e inserir os ponteiros para as subárvores correspondentes a `add` (`add2`) e `term` como filhos desse nó. Esses ponteiros se encontram, respectivamente, nas posições \$1 e \$3 da pilha semântica.
- Na segunda produção, o ponteiro para subárvore correspondente a `term` é copiado para o atributo \$\$ de `add`.

```
add      : add '+' term {
            $$=CriaOperador(Mais, numLinha,$1,$3, NULL);
        }
        | term { $$=$1; }
```

Ações semânticas correspondentes às rel-produções

- As ações semânticas para as duas primeiras rel-produções são semelhantes e envolvem a criação de um nó para representar seus operadores ('<' e LEQ) e inserir os ponteiros para as subárvores correspondentes. O ponteiro para o nó criado é repassado como atributo de rel ao lado esquerdo das produções como o atributo \$\$.
- Para a terceira rel-produção a ação semântica consiste em propagar o ponteiro para a subárvore correspondente a add para o atributo do símbolo rel no momento da redução.
- As produções e as respectivas ações semânticas são mostradas abaixo:

```
rel      : rel  '<'  add
{ $$ = CriaOperador(Menor, numLinha, $1, $3, NULL); }
  | rel LEQ add
{ $$ = CriaOperador(MenorIgual, numLinha, $1, $3, NULL); }
  | add
                                     { $$ = $1; }
  ;
```

- As ações semânticas para as stmt-produções são semelhantes às utilizadas nos não-terminais nos slides anteriores e são mostradas a seguir:

```
stmt      : expr ';'                               { $$ = $1; } =
           | IF '(' expr ')' stmt
{ $$ = CriaOperador(If, numLinha, $3, $5, NULL); }
           | WHILE '(' expr ')' stmt
{ $$ = CriaOperador(While, numLinha, $3, $5, NULL); }
           | DO stmt WHILE '(' expr ')'
{ $$ = CriaOperador(Do, numLinha, $2, $5, NULL); }
           | block                                { $$ = $1; }
           ;
```

Ações semânticas correspondentes às stmts-produções

- As stmts-produções são utilizadas para representar uma lista de comandos que termina com a cadeia vazia:

```
stmts : stmt stmts | /*cadeia vazia/*
```

A lista de comandos é representada por uma subárvore binária, formada por nós com TipoOperador (ver struct que define o nó) igual a 1stStmt. O primeiro nó 1stStmt tem como primeiro filho a subárvore correspondente ao primeiro stmt e como segundo filho, outro nó 1stStmt que por sua vez tem como primeiro filho, a subárvore correspondente ao segundo stmt, e assim, por diante. Esse arranjo da lista de comando é obtida no seguinte trecho do esquema de tradução:

```
stmts      :      stmt stmts
{ $$ = CriaOperador(1stStmt , numLinha, $1, $2, NULL); }
  | /*cadeia vazia */      { $$ = NULL; }
;
```

Ações semânticas correspondentes a block-produção e a program-produção

- Um programa em SimpleLang consiste de uma lista de stmt. Portanto, a ação semântica na block-produção tem como objetivo apenas passar o ponteiro para a subárvore binária que representa a lista de comandos como um atributo de block:

```
block      :      '{' stmts '}'      {$$=$2;}  
          ;
```

- A ação semântica para a program-produção tem como objetivo armazenar o ponteiro para o nó 1stStmt que corresponde ao primeiro nó da lista de comandos, em uma variável global de nome raiz:

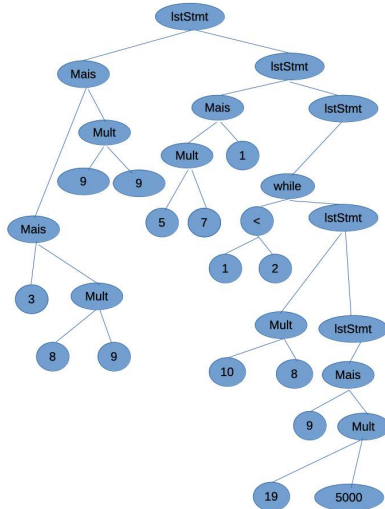
```
program    :      block      {raiz=$1;}  
          ;
```

Exemplo

Árvore abstrata correspondente

Programa em SimpleLang

```
{  
  3+8*9+(9*9);  
  5*7+1;  
  while (1<2) {  
    10*8;  
    9+(19*5000);  
  }  
}
```



Arquivos com códigos-fontes

- No site da disciplina na plataforma Turing, no Bloco 2, há uma pasta compactada com todos os arquivos correspondentes ao exemplo anterior.
- Na pasta há os seguintes arquivos:
 - Um arquivo de entrada para o Flex (`lexico.l`);
 - Um arquivo de entrada para o Bison (`simpleLang.y`) com o esquema de tradução;
 - O arquivo `tipos.h` que contém a descrição do tipo de dados de cada nó da árvore abstrata,
 - O arquivo `simpleLang.h`, gerado pelo Bison, usando a opção `-d`: `bison -d -osimpleLang.c simpleLang.y`. Esse arquivo contém a definição do tipo da pilha semântica em C e a definição das constantes inteiras positivas que correspondem aos tokens da linguagem. Esse arquivo é incluído no analisador léxico (através do comando `#include` do pré-processador).
 - `Makefile` para compilar os fontes e gerar o código executável.